

Academy Frontend

This is a [Next.js](#) project bootstrapped with `create-next-app`, interacting with a Frappe backend.

Getting Started

First, run the development server:

```
npm run dev
# or yarn dev / pnpm dev / bun dev
```

Open <http://localhost:3000> with your browser to see the result.

Architecture & Folder Structure

The project has been refactored for better separation of concerns, scalability, and security.

API Layer Structure (`/api`)

All API calls and session logic have been centralized into the `/api` directory:

```
api/
├── api-routes.ts    ← Single source of truth for all Frappe backend endpoints
├── api.ts           ← Client-side API caller. Wraps clientFetch to send requests
├── api-server.ts    ← Server-side API caller. Forwards cookies using `next`
├── auth.ts          ← SSR security guard (`requireAuth()`). Validates session
└── session.ts       ← Core session logic. Queries Frappe to get user profile
```

Library & Utilities (`/lib`)

The `lib` folder strictly contains formatting and utilitarian functions:

```
lib/
├── client-fetcher.ts ← Client-side fetch wrapper. Auto-includes credentials
```

- └─ `server-fetcher.ts` ← Server-side fetch wrapper. Reads cookies from `ne`
 - └─ `utils.ts` ← General utilities (e.g., Tailwind `cn()` merger).
 - └─ `excel-export.ts` ← Excel formatting & export utility.
-



Authentication Flow

The authentication system is built for **security, performance, and multi-tab synchronization**.

1. Login Process:

- User enters credentials on the login page.
- `api.login()` hits the Frappe backend, which sets an `HttpOnly sid` cookie.
- The client calls `api.getLoggedInUser()` to hydrate the user state in React Context (`AuthContext`).
- A non-httpOnly `role` cookie is set purely for fast edge-middleware routing.
- The login event is broadcasted via `BroadcastChannel` to update other open tabs instantly.

2. Middleware Fast-Routing (UX layer):

- `middleware.ts` runs on the Edge.
- It checks the presence of the `sid` and `role` cookies to do immediate, lightweight redirects (e.g., blocking non-admins from hitting `/dashboard`).
- *No backend calls are made here* to keep responses blazing fast.

3. SSR Security Guard (`api/auth.ts`)(Security layer):

- Protected routes (e.g., `/my-bookings`, `/book`, and admin layouts) enforce security by calling `await requireAuth()`.
- `requireAuth()` executes `session.ts`, which makes a real-time validation call to the Frappe backend using the `sid` cookie.
- The result is cached with React `cache()`, meaning a single request (e.g., Layout + Page both requiring auth) guarantees only **one API hit**.

4. Multi-Tab Sync:

- Handled inside `AuthProvider` (`context/auth-context.tsx`).
 - When a user logs in on Tab A, Tab B dynamically refetches the session and re-renders server components via `router.refresh()`.
 - When a user logs out, all tabs are immediately cleared and redirected to `/login`.
-

Other Features

- **Public vs Protected Routes:** Public marketing pages (`/` , `/about` , `/calendar`) do not require authentication. The `(user)` router layout dynamically allows mixed public and protected pages by executing `requireAuth()` only on specific page endpoints.
- **Centralized API Routes:** No hardcoded absolute URLs scattered throughout the components.
- **Robust Error Handling:** Fetch wrappers intelligently parse Frappe's `_server_messages` format to extract meaningful, human-readable errors for the UI.